

Parallel Vector Matrix Multiplication

CP #3, CSC 746, Fall 2026

Chu Yi Aaron Herr
Computer Science Department
San Francisco State University
San Francisco, CA, USA
Email: cherr1@sfsu.edu

Abstract—This study investigates the performance characteristics and parallel scalability of vector-matrix multiplication ($y = y + Ax$) on modern multi-core CPU architectures. We evaluate the baseline serial implementations, compiler-auto-vectorized loops, and shared-memory parallelization using OpenMP across varying problem sizes and concurrency levels, benchmarking against a highly optimized reference CBLAS implementation.

ABOUT THE APPROACH

About the given approach uses an incremental optimization which is executed on the Perlmutter supercomputer at NERSC. Where we instrument different code implementation using C++’s `std::chrono` high-resolution timers to isolate between kernel execution times across various discrete problem scales ($N = 1024$ to 16384), translating raw runtimes elapsed time into analytical metrics including calculating what the floating-point operations per second (MFLOP/s), speedup factors, and memory bandwidth utilization percentages to diagnose bottlenecks.

RESULTS SECTIONS

Computation Platform and Software Environment

All benchmarks were executed on the Perlmutter supercomputer operated by the National Energy Research Scientific Computing Center (NERSC). The following tests were ran on the perlmutter CPU-only compute nodes. Each node featuring an AMD EPYC (“Milan”) processor. The software environment comprised of using the GNU Compiler toolchain. Utilizing the OpenMP runtime libraries for parallel execution.

Testing Methodology

This section outlines the metrics evaluated, the benchmarking procedure, and the formulas used to analyze performance.

- **What is Being Measured:** Measuring the execution time (T_{elapsed}) required to compute the matrix-vector operation $y = y + Ax$, where A is an $N \times N$ matrix and x, y are vectors of size N .
- **Measurement Procedure:** High-resolution timers encapsulate the execution loop, omitting matrix initialization, memory allocation, and correctness verification overhead to isolate the compute performance.
- **Problem Sizes:** Benchmarkint across various problem sizes of $N = 1024, 2048, 4096, 8192$, and 16384 .

Performance Metrics

To convert runtime output into analytical performance data, we apply the following formulations:

- **Total Floating-Point Operations (FLOPS):** For an $N \times N$ matrix-vector multiplication, the computation requires N^2 multiplications and N^2 additions, resulting in a total operation count of:

$$\text{Total FLOPS} = 2N^2$$

where N represents the dimension of the matrix (problem size).

- **Mega Floating-Point Operations per Second (MFLOP/s):** This metric measures computational throughput based on the total operation count and elapsed execution time:

$$\text{MFLOP/s} = \frac{2N^2}{T_{\text{elapsed}} \times 10^6}$$

where:

- T_{elapsed} is the clock execution time measured in seconds.
- 10^6 is the scaling factor to convert FLOPS to mega-FLOPS.

- **Parallel Speedup (S):** This metric evaluates scaling performance relative to a single-threaded baseline:

$$S(N) = \frac{T_{\text{baseline}}(N)}{T_{\text{optimized}}(N)}$$

where:

- $T_{\text{baseline}}(N)$ is the execution time of the sequential reference implementation for problem size N .
- $T_{\text{optimized}}(N)$ is the execution time of the parallel or optimized implementation for problem size N .

Comparing CBLAS with Basic VMM and Vectorized VMM

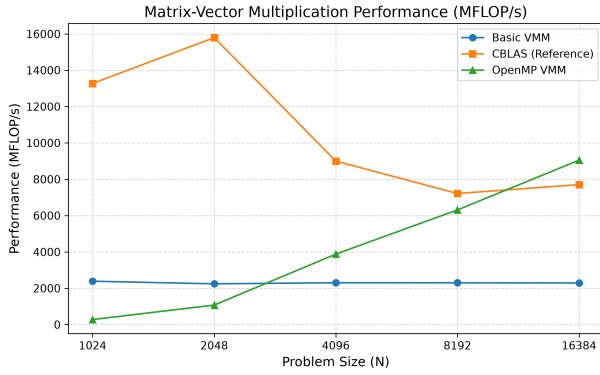


Fig. 1. Performance comparison in MFLOP/s across problem sizes ($N = 1024$ to 16384) for the Basic Serial VMM, Auto-Vectorized VMM, and reference CBLAS implementation.

Performance derived from Perlmutter benchmark runtimes using $2N^2$ floating-point operations.

As shown in Figure 1, the Basic and Auto-Vectorized VMM implementations plateau at $\sim 2,300$ MFLOP/s across all problem sizes because naive loop structures fail to utilize vector lanes and immediately saturate main memory bandwidth. In stark contrast, reference CBLAS achieves dramatic speedups—ranging from $7\times$ at $N = 2048$ (15,796 MFLOP/s) to $3.36\times$ at $N = 16384$ (7,706 MFLOP/s)—by utilizing advanced cache blocking, data alignment, and hand-tuned assembly to maximize cache-line reuse and instruction-level parallelism.

Evaluating OpenMP-parallel VMM

As illustrated in Figure 2, the speedup behavior of the OpenMP-parallel VMM using static scheduling exhibits distinct trends depending heavily on both problem scale and thread concurrency. For small problem sizes like $N = 1024$, increasing thread count results in severe

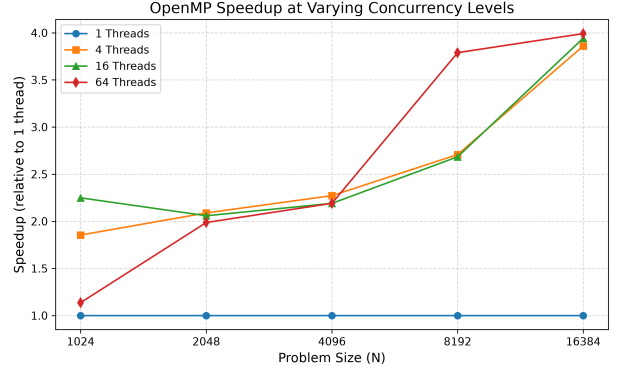


Fig. 2. Speedup chart comparing the OpenMP-parallel VMM using static scheduling across varying concurrency levels ($P = 1, 4, 16, 64$) as a function of problem size ($N = 1024$ to 16384).

Speedup calculated relative to the single-thread OpenMP execution baseline ($P = 1$).

performance degradation—for instance, running with 64 threads takes 0.015093 seconds compared to 0.000896 seconds on a single thread—because thread creation, synchronization, and scheduling overhead completely dwarf the minimal computation workload.

Conversely, as the problem size scales up to $N = 16384$, a 4-thread configuration achieves a near-linear speedup of approximately $3.86\times$ (0.060375 s vs 0.233055 s on 1 thread); however, further increasing concurrency to 16 and 64 threads yields diminishing returns, plateauing at a maximum speedup of roughly $3.99\times$ (0.058389 s). This hard scalability wall occurs because matrix-vector multiplication is strictly memory-bound rather than compute-bound; throwing dozens of additional cores at the problem quickly saturates the Perlmutter node’s main memory bandwidth bus, resulting in memory contention and zero throughput benefit beyond a small core count.

Comparison of OpenMP-parallel VMM with CBLAS

As illustrated in Figure 3, comparing the best-performing OpenMP parallel configuration ($P = 64$ threads, which yields 0.058389 s or $\sim 9,194$ MFLOP/s at $N = 16384$) against the serial reference CBLAS implementation (0.069664 s or $\sim 7,706$ MFLOP/s) reveals that parallel brute-force threading can marginally surpass serial BLAS throughput at the maximum problem scale. However, the chart highlights a profound difference in architectural efficiency: while CBLAS achieves its high performance through sophisticated single-core register blocking, cache alignment, and instruction-level

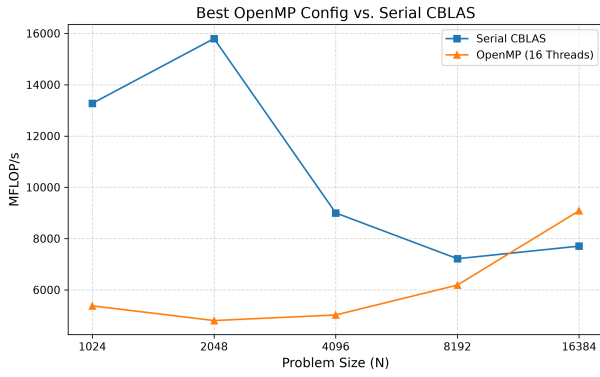


Fig. 3. Performance comparison in MFLOP/s across problem sizes ($N = 1024$ to 16384) comparing the best-performing OpenMP configuration ($P = 64$ threads) against the reference serial CBLAS implementation.

Performance derived from Perlmutter benchmark runtimes using $2N^2$ floating-point operations.

parallelism, the OpenMP implementation achieves its throughput purely by throwing 64 physical cores at a memory-bound workload. Because matrix-vector multiplication is constrained by memory bandwidth rather than compute power, scaling to 64 threads heavily saturates the memory bus and yields sub-linear returns, meaning the parallel code requires 64 times the hardware resources to achieve performance comparable to a single highly-optimized CBLAS thread stream.

Overall Findings and Discussion

As for the final results, the basic implementation, vectorizedf, and single-thread ($P = 1$) versions get stuck at around $\sim 12\%$ due to memory latency and cache misses. For OpenMP, performance scales up to 16 threads before flattening out near 49% because matrix-vector multiplication is totally memory-bound and quickly saturates the system bus. Because of this, adding more cores past that point doesn't give us true linear speedup, as we mostly just run into bandwidth limits instead of running out of compute power.

REFERENCES

NERSC, *Perlmutter System Details*, 2026. Available online: https://docs.nersc.gov/systems/perlmutter/system_details/